

UD 10. COLECCIONES DE DATOS

Programación – 1º DAM
Curso 2025/2026

- **Introducción**
- **Interfaz Collection**
 - **Breve presentación de las listas**
 - **Métodos básicos de la interfaz Collection**
- **Métodos específicos de la interfaz List**

Introducción

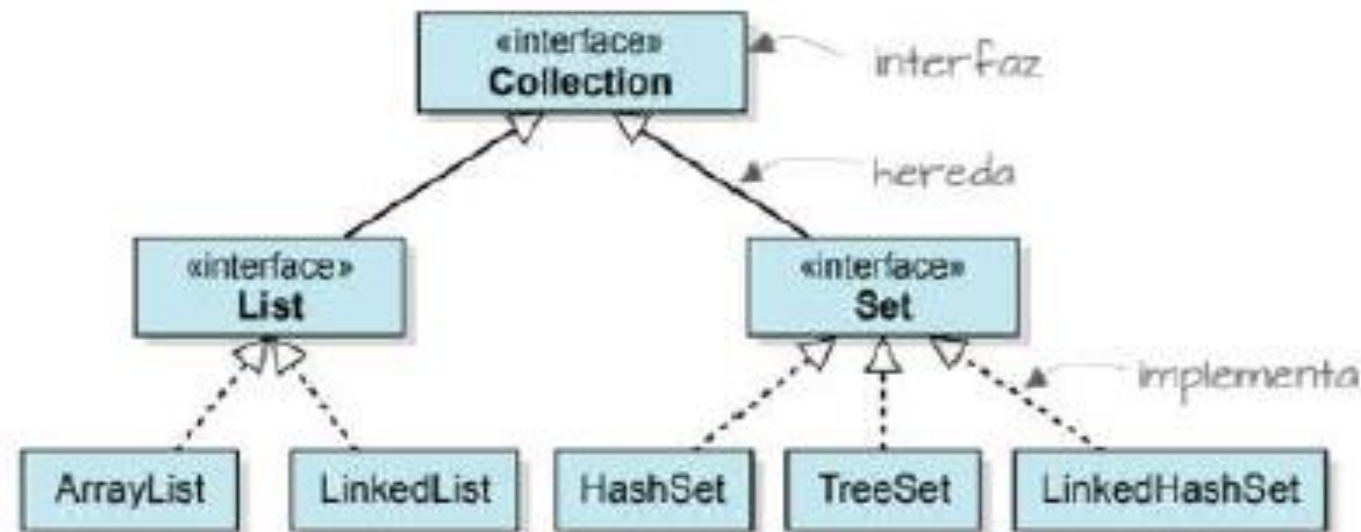
A menudo necesitamos guardar información, pero no sabemos de antemano el espacio que va a ocupar en la memoria. En estos casos, los arrays no son la solución adecuada ya que su tamaño debe permanecer fijo una vez creados. Al redimensionarlos, lo que hacemos es crear otros nuevos y copiar todos los datos del antiguo, con la sobrecarga que ello supone para la gestión de memoria.

En su lugar, necesitamos estructuras dinámicas de datos, es decir, objetos que alberguen datos que se puedan insertar y eliminar, cambiando el tamaño de la estructura sin alterar los datos restantes, todo ello en tiempo de ejecución.

Introducción

Para este fin, Java nos proporciona una serie de estructuras dinámicas que comparten un conjunto de métodos declarados en la interfaz ***Collection***.

Todas ellas implementan dicha interfaz, aunque de distinta forma.



Introducción

Una colección es un objeto que sirve para agrupar un conjunto de objetos, llamados ***elementos***, que generalmente guardan una relación con ellos. Los métodos de las colecciones nos permiten llevar a cabo distintas operaciones con sus elementos, como la inserción, la eliminación, la búsqueda o la ordenación.

Hay colecciones de distintos tipos, adaptadas a distintos fines más o menos específicos. Es por esto que existe una jerarquía de interfaces que se implementan en distintas clases, y un conjunto de algoritmos que permiten la manipulación de los datos almacenados en las colecciones.

Introducción

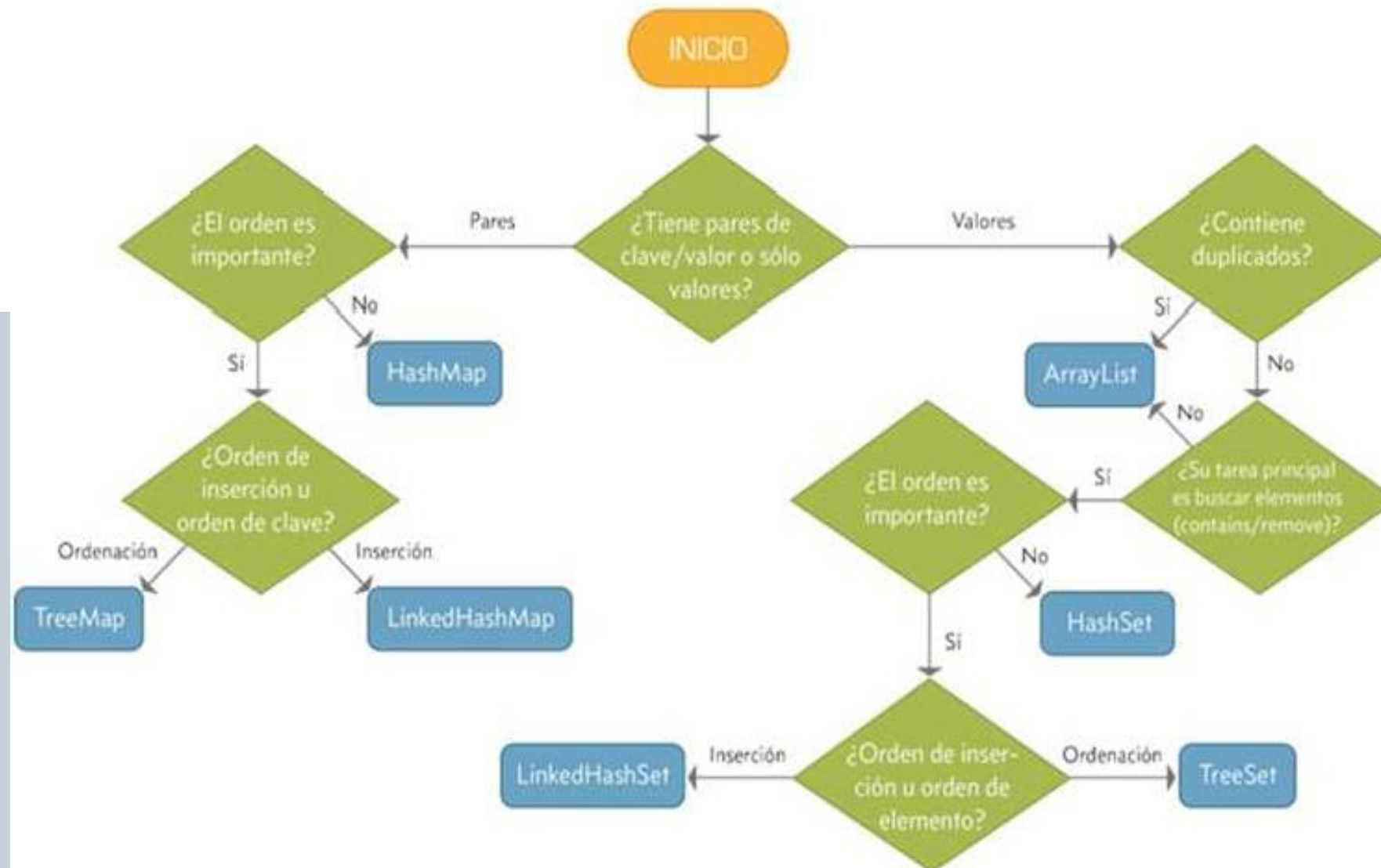
Los tipos fundamentales de estructuras son tres:

- **Listas**: responden a la necesidad de manejar sucesiones de **datos** que pueden estar **repetidos** y cuyo **orden** puede ser **relevante**. Podemos insertar o eliminar datos en ellas sin limitaciones. Implementan la interfaz **List** que, al heredar de **Collection**, incorpora todas sus funcionalidades y añade alguna más.
- **Conjuntos**: el **orden** de los datos **no es relevante** y lo que realmente importa es la simple **pertenencia a la estructura**, con lo que las **repeticiones no son posibles** ni tienen sentido. Implementan la interfaz **Set**, que también hereda de **Collection**.

Introducción

- **Mapas o diccionarios**: aunque están dentro del marco de trabajo de las colecciones, **no implementan la interfaz `Collection`**. Sirven para guardar datos identificados por claves que no se repiten. Implementan la interfaz ***Map***, que **no hereda de *Collection***.

Introducción



Interfaz Collection

La interfaz **Collection** define las funcionalidades comunes a todas las colecciones, ya sean listas o conjuntos. Sin embargo, las clases de la API que la implementan son listas o conjuntos, es decir, implementan la interfaz **List** o **Set**, que son extensiones de **Collection**.

Métodos básicos de la interfaz Collection

Los métodos de la interfaz Collection son de dos tipos. Unos afectan a **elementos individuales (métodos básicos)** y otros a **grupos de elementos (métodos globales)**.

Métodos básicos:

- **Método de inserción:** para añadir elementos nuevos en una colección.
 - ***boolean add(E elem)***: recibe el objeto que se va a insertar. Si la inserción es correcta, devuelve true.

Si la colección es una lista, el nuevo elemento siempre se insertará, y además, lo hará al final.

Ejemplo:

```
Cliente cliente = new Cliente("111", "Marta", "12/02/2000");  
coleccionClie.add(cliente);
```

Métodos básicos de la interfaz Collection

- **Métodos de eliminación:** para eliminar elementos en una colección.
 - ***boolean remove(Object ob)***: eliminar un elemento ob de una colección. Si está repetido, elimina solo el primero que encuentra. Devuelve true si la eliminación es correcta, false en caso contrario (por ejemplo, el objeto no estaba en la colección). Este método no exige a ob que sea del tipo genérico E con el que se ha declarado la colección, ya que no va a añadir ningún elemento, y no hay peligro de que se inserte un objeto de una clase no permitida.

Ejemplo:

Para eliminar a Marta, que la hemos insertado en el ejemplo anterior:

```
coleccionClie.remove(cliente);
```

Métodos básicos de la interfaz Collection

- ***void clear()***: nos permite eliminar todos los elementos de una colección y dejarla vacía. Es decir, la colección queda vacía y disponible para volver a insertar nuevos elementos.

Ejemplo:

```
coleccionClie.clear();
```

Métodos básicos de la interfaz Collection

- **Métodos de comprobación:** nos permiten comprobar el estado de una colección.
 - ***int size()***: devuelve el número de elementos (o nodos) insertados en una colección.
 - ***boolean isEmpty()***: permite saber si una colección está vacía. Devuelve true si está vacía.

```
coleccionClie.size();
```

- ***boolean contains(Object ob)***: nos dice si un elemento ob determinado está en una colección. Devuelve true si ob pertenece a la colección.

```
coleccionClie.contains(new Cliente("115", "Jorge", "16/03/1999"));
```

Métodos básicos de la interfaz Collection

En la búsqueda del objeto ob (llamado clave de búsqueda), **contains** usa el método **equals()** para compararlo con los elementos de la colección. Por ejemplo, si nuestro método equals solo tiene en cuenta el dni del cliente, el programa solo va a comparar este atributo de la clave de búsqueda con los distintos elementos de la colección.

```
coleccionClie.contains(new Cliente("115", "", ""));
```

Esto nos permite hacer búsquedas sin conocer ni tener que escribir toda la información de un elemento. Basta conocer el o los atributos que usa el método equals(). Si tenemos que hacer muchas búsquedas puede ser útil implementar un constructor de Cliente con el DNI como único parámetro. Así, la llamada a contains sería:

```
coleccionClie.contains(new Cliente("115"));
```

Métodos básicos de la interfaz Collection

- **Otros métodos:**
 - ***String toString()***: devuelve una cadena que representa la colección. Todas las colecciones tienen implementado este método, que muestra los elementos entre corchetes y separados por comas. Cada elemento se muestra, a su vez, según la implementación de `toString()` que tenga la clase E, en nuestro caso la clase `Cliente`. Por tanto, para mostrar los elementos de `coleccionClie`, escribimos:

```
System.out.println(coleccionClie);
```

Métodos básicos de la interfaz Collection

A menudo necesitamos recorrer una colección elemento a elemento. Una forma de hacerlo es con iteradores, que son objetos que van apuntando sucesivamente a los elementos de la colección, empezando por el primero. Para usar un iterador con una colección concreta, primero hay que crearlo.

- ***Iterator<E> iterator()***: invocado por la colección, nos devuelve un iterador asociado a ella. Aquí E será del mismo tipo que el de la colección.

```
Iterator<Cliente> it = coleccionClie.iterator();
```

Siendo it el iterador que sirve para recorrer coleccionClie. Para ello dispone de los métodos `hasNext()` y `next()`, que se complementan y emplean conjuntamente. Inicialmente it apunta al inicio de la colección, justo antes del primer elemento.

Métodos básicos de la interfaz Collection



- ***boolean hasNext()***: comprueba si hay un elemento siguiente, es decir, si quedan elementos por visitar, y nos devuelve true o false, según el caso.
- ***E next()***: el iterador avanza al siguiente elemento, si existe, y nos lo devuelve. En caso de no haya siguiente, porque estemos al final de la colección o porque esté vacía, **next()** lanzará la excepción ***NoSuchElementException***. La primera llamada a next() devuelve el primer elemento de la colección si esta no está vacía.

Para evitar la excepción, los dos métodos se usan juntos, de firma que solo se llama al método next() si antes se ha comprobado, con hasNext() que hay elemento siguiente.

Métodos básicos de la interfaz Collection

Ejemplo:

```
Iterator<Cliente> it = coleccionClie.iterator();  
while(it.hasNext()) {  
    Cliente p = it.next();  
    System.out.println(p);  
}
```



Métodos básicos de la interfaz Collection

En el bucle hay que observar varias cosas. En primer lugar, la declaración e inicialización del iterador **it** se podría haber incluido en la parte de inicialización del bucle **for**. En segundo lugar, la parte de los incrementos está vacía. Esto se debe a que el método **next()** incrementa automáticamente el iterador para que apunte al siguiente elemento y después lo devuelve.



Métodos básicos de la interfaz **Collection**

Los iteradores tienen un tercer método que permite eliminar elementos de una colección.

- ***void remove()***: elimina de la colección el último elemento devuelto por `next()`, que es el apuntado por el iterador en cada momento. Así podremos eliminar un elemento de una colección dependiendo de alguna condición mientras se está recorriendo un iterador. Esta es la única forma de eliminar un elemento preservando la integridad de la colección. Aunque tenga el mismo nombre, no debemos incluirlo en el método ***remove(Object)*** de la interfaz **Collection**, que se invoca desde un objeto colección y no debe usarse dentro del bucle de un iterador. En este contexto, debemos invocar el del iterador, al que no se le pasa ningún parámetro.

Métodos básicos de la interfaz Collection

Antes comentábamos que la única forma de eliminar un elemento de una colección mientras se recorre preservando la integridad es con el iterador. Esto se debe a que, si se hace de cualquier otra forma (con un for-each o un for con get(index)), nos lanzaría una **ConcurrentModificationException**. Java lanza este error porque la colección se está modificando mientras se recorre.

Métodos básicos de la interfaz Collection

```
Iterator<Cliente> it =  coleccionClie.iterator();
while(it.hasNext()) {
    Cliente p = it.next();
    if(p.fechaNacimiento.compareTo(LocalDate.of(2000, 1, 1))<0) {
        it.remove();/*elimina p, último cliente devuelto por next()
        ;No usar coleccionClie.remove(p)!*/
    }
}
```

Métodos básicos de la interfaz Collection

Una forma mucho más simple de recorrer una colección es por medio de la estructura **for extendido** o **for-each**.

```
for (Cliente c : coleccionClie) {  
    System.out.println (c);  
}
```

que sería algo así como: “para cada cliente c que pertenece a coleccionClie, mostrar c”. En este bucle, la variable local c, de tipo Cliente, va tomando todos los valores de la colección. Sin embargo, hay operaciones para las que el bucle for-each no vale. Por ejemplo, no podemos eliminar elementos, ya que c es siempre la referencia a una copia de un elemento de la colección. Para la eliminación de elementos durante el recorrido de una colección, tendremos que usar un iterador.

Métodos básicos de la interfaz Collection

Como hemos visto antes, hay situaciones en las que se recomienda el uso de Iteradores porque el recorrido de una colección con un for normal y usando el método `get(i)`, y el recorrido usando for-each pueden dar fallo. Explicamos el porqué:

- Fallo usando for-each.
- Fallo usando for y `get(i)`

Métodos básicos de la interfaz Collection

Fallo usando for-each. Se produce cuando necesitas eliminar elementos mientras recorres la lista.

Esto lanzará el error **ConcurrentModificationException**.

Ejemplo de fallo con for-each:

Ejemplo de fallo con for-each	Ejemplo de éxito con iterator
<pre>for (String e : lista) { if (e.equals("borrar")) { lista.remove(e); } }</pre>	<pre>Iterator<String> it = lista.iterator(); while (it.hasNext()) { String elemento = it.next(); if (elemento.equals("borrar")) { it.remove(); } }</pre>

Usar Iterator siempre que quieras **modificar (especialmente eliminar)** durante la iteración.

Métodos básicos de la interfaz Collection

Fallo usando for y get(i). En este caso no siempre salta la excepción, pero hay un problema peor: **te saltas elementos**. Cuando eliminas un elemento, los índices cambian:

Ejemplo: ["A", "borrar", "borrar", "C"]

Paso a paso:

1. $i = 1 \rightarrow$ eliminas "borrar"
2. La lista queda: ["A", "borrar", "C"]
3. $i++ \rightarrow i = 2$
4. Te saltas el segundo "borrar"

Resultado: **no eliminas todos los elementos que querías**

Métodos básicos de la interfaz Collection

Las soluciones para el problema anterior serían:

- **Opción 1: recorrer hacia atrás para evitar problemas de desplazamiento**

```
for (int i = lista.size() - 1; i >= 0; i--) {  
    if (lista.get(i).equals("borrar")) {  
        lista.remove(i);  
    }  
}
```

- **Opción 2: ajustar el índice manualmente**

```
for (int i = 0; i < lista.size(); i++) {  
    if (lista.get(i).equals("borrar")) {  
        lista.remove(i);  
        i--; // compensas el desplazamiento  
    }  
}
```

LISTAS

Las listas son clases que implementan la interfaz List, y sirven para almacenar datos que se pueden repetir y cuyo orden de inserción puede ser relevante. Hay dos implementaciones de **List**: las clases **ArrayList** y **LinkedList**.

Las dos implementan los mismos métodos y funcionalidades.

La diferencia entre ArrayList y LinkedList radica en la implementación interna y solo afecta levemente al rendimiento.

- La primera, **ArrayList**, es más rápida en las operaciones que supongan recorrer la lista para la lectura o modificación de elementos.
- La segunda, **LinkedList**, tiene mejor rendimiento en las operaciones de inserción y eliminación de elementos.

Usaremos la primera en los ejemplos, aunque podríamos cambiar ArrayList por LinkedList sin alterar ningún resultado.

LISTAS

La sintaxis para construir un objeto `ArrayList` con un tipo genérico de datos `E` (Cliente, Empleado, Integer, etc) es:

```
ArrayList<E> lista = new ArrayList<>();
```

O bien, de forma más general, dado que tanto la clase **ArrayList** como **LinkedList** implementan todos los métodos de la interfaz **List**, es posible utilizar una variable de este tipo para referenciar objetos de ambas clases.

```
List<E> lista = new ArrayList<>();
```

En esta lista solo se podrán insertar objetos (elementos) del tipo **E**.

LISTAS

La construcción de un **ArrayList** que nos sirva para guardar objetos de tipo **Cliente** será:

```
List<Cliente> listaClientes = new ArrayList<>();
```

Una vez creada la colección **listaClientes**, disponemos de una estructura dinámica donde insertar o eliminar objetos **Cliente**. Antes vamos a definir una clase **Cliente** que nos permita probar los distintos métodos con ejemplos concretos:

LISTAS

```
public class Cliente implements Comparable<Cliente>{
    String dni, nombre;
    LocalDate fechaNacimiento;
    Cliente (String dni, String nombre, String fechaNacimiento){
        this.dni=dni;
        this.nombre=nombre;
        DateTimeFormatter formatoFechas= DateTimeFormatter.ofPattern("dd/MM/yyyy");
        this.fechaNacimiento=LocalDate.parse(fechaNacimiento, formatoFechas);
    }
    int edad() {
        return (int)fechaNacimiento.until(LocalDate.now(), ChronoUnit.YEARS);
    }
    @Override
    public boolean equals(Object ob) {
        return dni.equals(((Cliente)ob).dni);
    }
    @Override
    public int compareTo(Cliente otro) {
        return dni.compareTo(otro.dni);
    }
    @Override
    public String toString() {
        return "DNI: "+dni+" Nombre: "+nombre+" Edad: "+edad()+"\n";
    }
}
```

LISTAS

Para estudiar la interfaz **Collection**, dado que la interfaz **List** hereda de ella, vamos a referenciar la lista **listaClientes** con la variable **coleccionClie** del tipo **Collection**. Con ello conseguiremos tener acceso únicamente a las funcionalidades de **Collection**, que son las que vamos a estudiar ahora, y no a las específicas de la interfaz **List**, que estudiaremos después.

```
Collection<Cliente> coleccionClie = listaClientes;
```

Mientras usemos la variable **coleccionClie**, el objeto referenciado (una lista) se comportará como una simple colección, sin ninguna de las particularidades específicas de las listas.



Métodos específicos de la interfaz List

Todos los métodos vistos hasta ahora pertenecen a la interfaz Collection, es decir, son implementados por todas las colecciones, tanto listas como conjuntos.

En realidad, **las listas implementan la interfaz List, que hereda de Collection, añadiéndole una serie de métodos y funcionalidades específicas**, que no comparten los conjuntos.

La funcionalidad más importante exclusiva de las listas (ArrayList o LinkedList) es el **acceso posicional a sus elementos por medio de índices**. El primer elemento tiene índice 0, el segundo índice 1 y así sucesivamente.

Los elementos de las listas pueden estar repetidos y el orden en el que se encuentran es relevante.

Métodos específicos de la interfaz List

Ejemplo:

```
List<Integer> listaEnteros = new ArrayList();  
//insertamos algunos elementos  
listaEnteros.add(3);  
listaEnteros.add(1);  
listaEnteros.add(-2);  
listaEnteros.add(0);  
listaEnteros.add(3);  
listaEnteros.add(7);  
//mostramos la lista  
System.out.println(listaEnteros); //[3, 1, -2, 0, 3, 7]
```

En la lista, el **método add()** inserta el nuevo elemento al final.

Métodos específicos de la interfaz List

Los métodos más importantes aportados por la interfaz **List** son:

- **E get (int indice):** devuelve el elemento que ocupa el lugar **indice** en la lista siendo 0 el índice del primer elemento.

Ejemplo:

```
listaEnteros.get(2); //devolverá -2, que es el valor del  
elemento que ocupa el tercer lugar de la lista.
```

Métodos específicos de la interfaz List

- **E set (int índice, E elem):** guarda el elemento **elem** en la posición **índice**, machacando el valor que hubiera previamente en esa posición, que es devuelto.

Ejemplo:

Con el siguiente código, vamos a poner el valor 10 en el elemento de índice 3, sustituyendo su valor actual (0), que es devuelto y asignado a la variable **y**.

```
Integer y = listaEnteros.set(3, 10);  
System.out.println("y: " + y);  
System.out.println(listaEnteros);
```

Se mostrará por pantalla:

```
y: 0  
[3, 1, -2, 10, 3, 7];
```

Métodos específicos de la interfaz List

- **void add (int índice, E elem):** inserta el valor **elem** en la posición **índice**. Todos los elementos que ocupaban una posición igual o mayor que índice, se desplazan una posición hacia el final de la lista, para dejar hueco al nuevo elemento. Por ejemplo, si queremos insertar el valor 5 entre el -2 y el 10, se insertará en la posición 3, que actualmente ocupa el 10. Este y los elementos que le siguen se desplazarán un lugar hacia el final de la lista.

```
listaEnteros.add(3, 5);
```

```
System.out.println(listaEnteros);
```

```
System.out.println(listaEnteros);
```

Se mostrará por pantalla:

```
[3, 1, -2, 5, 10, 3, 7];
```

Métodos específicos de la interfaz List

- **void addAll (int índice, Collection<? extends E> c):** inserta todos los elementos de la colección c, en el mismo orden que tengan, en la lista que invoca al método, empezando por el lugar **índice** y desplazando hacia el final todos los elementos de la lista original a partir de **índice**, incluido este, tanto lugares como sean necesarios. Los elementos que queremos añadir deben coincidir en tipo con los de la lista donde se quieren añadir.

```
//creamos una segunda lista de enteros Integer  
ArrayList<Integer> otrosEnteros = new ArrayList<>();  
otrosEnteros.add(20);  
otrosEnteros.add(30);  
otrosEnteros.add(40);
```

Métodos específicos de la interfaz List

//ahora insertaremos esta lista en el lugar de índice 2 de listaEnteros, es decir, donde se encuentra el valor -2. Este elemento, junto con los que le siguen se desplazan hacia el final para hacer sitio a los tres valores insertados.

```
listaEnteros.addAll(2, otrosEnteros);  
System.out.println(listaEnteros);
```

Se mostrará por pantalla:

```
[3, 1, 20, 30, 40, -2, 5, 10, 3, 7];
```

Métodos específicos de la interfaz List

Para eliminar un elemento hay una versión sobrecargada del método **remove()**, que ya conocíamos, de la interfaz **Collection**.

- **E remove (int indice):** elimina el elemento que ocupa el lugar **indice** y lo devuelve.

En este caso, hay que tener cuidado si lo usamos en una lista de objetos Integer, como la de nuestro ejemplo, ya que una sentencia como:

```
listaEnteros.remove(5);
```

no será interpretada como que queremos eliminar un elemento **Integer** con valor 5 de la lista, es decir, el elemento de valor -2, ya que, al ser de tipo **int** el valor pasado como parámetro, dado que hay una versión de **remove()** que admite un **int** como argumento, Java le da prioridad y no hace autoboxing

Métodos específicos de la interfaz List

Autoboxing en Java es el proceso automático por el cual el compilador convierte un tipo primitivo (int, double, char, etc.) en su clase envolvente correspondiente (Integer, Double, Character, etc.).

Por tanto, en caso de que quisiéramos eliminar un elemento Integer cuyo valor es 5, escribiríamos:

```
listaEnteros.remove(Integer.valueOf(5));
```

Así estaríamos pasando como argumento un objeto Integer y no un valor int, con lo cual Java ejecuta la versión del método correspondiente a la interfaz Collection y elimina el elemento de valor 5.

Métodos específicos de la interfaz List

Además de los métodos de lectura, escritura, inserción y eliminación de elementos, heredados de la interfaz **Collection**, la interfaz **List** añade funciones de búsqueda, ordenación y comparación.

- **int indexOf (Object ob)**: devuelve el índice de la primera ocurrencia de ob en la lista, Si no está, devuelve -1.
- **int lastIndexOf(Object ob)**: igual que indexOf(), pero empezando la búsqueda por el final, devolviendo la última ocurrencia de ob.
- **boolean equals(Object otraLista)**: compara dos listas, tanto si las dos son ArrayList como si son LinkedList, o una de cada, y devuelve true si ambas tienen exactamente los mismos elementos, incluidas las repeticiones, en el mismo orden.

Métodos específicos de la interfaz List

- **void sort (Comparator<? super E> c):** ordena la lista invocante con el criterio de c, cuya implementación compara objetos de la clase E o una superclase (para que no se utilicen atributos que no están en E). Para ordenar por el criterio de orden natural de E, en caso de que exista, antes tendremos que obtener el comparador correspondiente, implementándolo nosotros mismos, o por medio del método **naturalOrder()**. También podremos recurrir a la clase **Collections** (con s al final, no confundir con la interfaz Collection).

Métodos específicos de la interfaz List

Ejemplo: Crea una lista de números enteros positivos introducidos por consola hasta que se introduzca uno negativo. A continuación, recorrer la lista y mostrar por pantalla los índices de los elementos de valor par, que será multiplicado por 100.

Métodos específicos de la interfaz List

```
public static void main(String[] args) {  
    List<Integer> lista=new ArrayList<Integer>();  
    System.out.println("Introducir número: ");  
    Integer n=new Scanner(System.in).nextInt();  
    while(n>0){  
        lista.add(n);  
        System.out.println("Introducir número: ");  
        n=new Scanner(System.in).nextInt();  
    }  
    System.out.println(lista);  
    System.out.println("Índices de valores pares: ");  
    for(int i=0;i<lista.size();i++) {  
        if(lista.get(i)%2==0) {  
            System.out.println(i+" ");  
            lista.set(i, lista.get(i)*100);  
        }  
    }  
    System.out.println("");  
    System.out.println(lista);  
}
```

Comparaciones en Java

Supongamos que tenemos una lista de objetos como Persona, o Producto, pero necesitamos ordenarlo ¿Cómo lo hacemos?

```
public static void main(String[] args) {  
  
    List<String> nombres = Arrays.asList("Carlos", "Ana", "Dionisio", "Bernardo");  
  
    //Es un modo de pasar de un Array estático a una List  
  
    System.out.println("lista original:" + nombres);//Muestra original  
  
    Collections.sort(nombres);  
  
    System.out.println("lista ordenada:" + nombres);//Ahora está ordenado  
  
}
```

Comparaciones en Java

La función **Collections.sort()** nos ordena una lista cualquiera.

Hay casos en que la propia estructura de datos los almacena de forma ordenada, como pasa en los TreeSet:

```
Set<String> otrosNombres = new TreeSet<String>();  
  
    otrosNombres.add("Mario");  
  
    otrosNombres.add("Fernando");  
  
    otrosNombres.add("Omar");  
  
    otrosNombres.add("Juana");  
  
System.out.println("conjunto ordenado:" + otrosNombres);
```

Comparaciones en Java

Funciona correctamente, porque **los elementos de las colecciones son comparables entre sí.**

Para que un objeto sea comparable, su clase debe implementar la interfaz **`java.lang.Comparable`**.

Esto quiere decir que, si queremos que una lista de objetos, o un conjunto, tenga sus elementos ordenados, y esos objetos son de una clase que hemos hecho, es necesario que nuestra clase implemente la interfaz ***`java.lang.Comparable`***.

Comparaciones en Java

Ejemplo:

```
public class Persona {

    private int idPersona;
    private String nombre;
    private int altura;

    public Persona() {
    }

    public Persona(int idPersona, String nombre) {
        this.idPersona = idPersona;
        this.nombre = nombre;
    }

    public Persona(int idPersona, String nombre, int altura) {
        this.idPersona = idPersona;
        this.nombre = nombre;
        this.altura = altura;
    }

    @Override
    public String toString() {
        return "Persona{" + "idPersona=" + idPersona
            + ", nombre=" + nombre + ", altura=" + altura + '}';
    }
}

public int getIdPersona() {
    return idPersona;
}

public void setIdPersona(int idPersona) {
    this.idPersona = idPersona;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public int getAltura() {
    return altura;
}

public void setAltura(int altura) {
    this.altura = altura;
}
```

Comparaciones en Java

1. Si queremos que los objetos se puedan almacenar en alguna colección ordenada, debemos (primero) **implementar la interfaz Comparable**.

```
public class Persona implements Comparable<Persona>{
```

2. Y sobrescribir el método **compareTo**:

```
@Override  
public int compareTo(Persona o) {  
    return this.nombre.compareTo(o.nombre);  
}
```

Comparaciones en Java

En el main se podrían añadir Personas a colecciones ya ordenadas:

```
public static void main (String[]args){  
    Set<Persona> personas = new TreeSet<Persona>();  
    personas.add(new Persona(1, "Mario"));  
    personas.add(new Persona(2, "Fernando"));  
    personas.add(new Persona(3, "Omar"));  
    personas.add(new Persona(4, "Juana"));  
}
```

O añadir las personas a una colección no ordenada y ordenarlas después.

```
ArrayList<Persona> listaPersonas = new ArrayList<Persona>();  
listaPersonas.add(new Persona(1, "Carla", 185));  
listaPersonas.add(new Persona(2, "Maria", 190));  
listaPersonas.add(new Persona(3, "Yovana", 170));  
Collections.sort(listaPersonas); // Ejemplo uso ordenación natural  
System.out.println("Personas Ordenadas por orden natural: " + listaPersonas);
```

Comparaciones en Java

Imaginemos que queremos ordenar a las Personas por un orden distinto a lo que hemos denominado orden natural, por ejemplo, por la altura.

Entonces tenemos que hacer uso de la interfaz **Comparator** de la siguiente manera:

```
public class PersonaPorAltura implements Comparator<Persona> {  
    @Override  
    public int compare(Persona o1, Persona o2) {  
        return o1.getAltura() - o2.getAltura();  
        // Devuelve un entero positivo si la altura de o1 es mayor que la de o2  
    }  
}
```

Comparaciones en Java

Y el uso en el programa principal sería el siguiente:

```
ArrayList<Persona> nuevaListaPersonas = new ArrayList<>();
nuevaListaPersonas.add(new Persona(1, "Carla", 185));
nuevaListaPersonas.add(new Persona(2, "María", 190));
nuevaListaPersonas.add(new Persona(3, "Yovana", 170));

nuevaListaPersonas.sort(new PersonaPorAltura());

//Se puede ordenar de dos modos
//Collections.sort(nuevaListaPersonas, new PersonaPorAltura());

System.out.println("Personas Ordenadas por orden total: " + nuevaListaPersonas);

nuevaListaPersonas.forEach(System.out::println);
```

Y la salida:

```
Personas Ordenadas por orden natural: [Persona{idPersona=1, nombre=Carla, altura=185}, Persona{idPersona=2, nombre=María, altura=190}, Persona{idPersona=3, nombre=Yovana, altura=170}]
Personas Ordenadas por orden total: [Persona{idPersona=3, nombre=Yovana, altura=170}, Persona{idPersona=1, nombre=Carla, altura=185}, Persona{idPersona=2, nombre=María, altura=190}]
Persona{idPersona=3, nombre=Yovana, altura=170}
Persona{idPersona=1, nombre=Carla, altura=185}
Persona{idPersona=2, nombre=María, altura=190}
```

Comparaciones en Java

Ahora la línea de ordenación `Collections.sort` lleva añadido el parámetro **new PersonaPorAltura()**.

Esta es otra forma del método **sort** de la clase **Collections**, en este caso pasándole como parámetros un objeto colección y un objeto que implemente **Comparator** y defina un orden total a utilizar.

Al pasarle un objeto de tipo **PersonaPorAltura** indicamos el orden total a utilizar que en este caso es la altura.

Comparaciones en Java

Conclusiones:

- Aunque **Comparable** y **Comparator** parecen iguales, en realidad no lo son, y mientras una se define para el orden natural, la otra se define para un orden total.
- El orden natural es utilizado por diversos métodos del api de Java, pero no siempre hemos de usar el orden natural. En determinados casos podremos querer ordenar objetos por un orden distinto al orden natural, y para ello nos será útil implementar la interface **Comparator**.
- La segunda conclusión es que gracias a la interfaz **Comparator**, podemos ordenar muy fácilmente colecciones utilizando clases que implementen el método **compare** por cada tipo de ordenación que deseemos.

MAPAS

Un mapa (**Map**) es una estructura de datos que almacena elementos de forma:

clave* → *valor

Cada clave (key) es única, no se permiten duplicados. Esa clave asociada a cada elemento nos permite recuperarlo

Cada clave apunta a un valor (value).e esta forma, podemos decir que para clave solo tenemos un valor.

Pertenece al paquete ***import java.util.Map;***

No extiende de *Collections*, pero forma parte del framework de colecciones.

MAPAS

Map es una interfaz y por tanto se deben implementar sus métodos. Las tres clases de Java que la implementan: **"HashMap"**, **"TreeMap"** y **"LinkedHashMap"**.

La diferencia entre ellas es la forma o el orden como guardan los valores.

- **HashMap**: los elementos que inserta no tendrán un orden específico. No aceptan valores nulos.
- **TreeMap**: los elementos tienen un orden (por clave, orden natural o comparador). Por ejemplo, si la clave son valores enteros los ordena de menor a mayor.
- **LinkedHashMap**: añade los elementos por orden de inserción, lo que hace que la búsqueda de elementos sea más lenta que el resto.

MAPAS

V put(K key, V value)	Inserta o actualiza un elemento
V get(Object key)	Devuelve el valor asociado a una clave
V remove(Object key)	Elimina un elemento por clave
boolean containsKey(Object key)	Comprueba si existe la clave
boolean containsValue(Object value)	Comprueba si existe el valor
Set<K> keySet()	Devuelve todas las claves
Collection<V> values()	Devuelve todos los valores
Set<Map.Entry<K,V>> entrySet()	Devuelve pares clave-valor
int size()	Número de elementos
boolean isEmpty()	Comprueba si está vacío
clear()	Elimina todos los elementos
V replace(K key, V value)	Reemplaza el valor de una clave

RECORRIDO DE UN MAPA

Un Map no se recorre directamente. Se hace a través de:

- `keySet()`
- `values()`
- `entrySet()`. Esta opción es la recomendada

Ejemplo:

```
for (Map.Entry<String, Integer> entry : mapa.entrySet()) {  
    System.out.println(entry.getKey()+"->" +entry.getValue());  
}
```

RECORRIDO DE UN MAPA

También se pueden usar **iteradores** que como en listas, nos permiten recorrer colecciones elemento a elemento.

En Map, se usan sobre:

- `keySet()`
- `values()`
- `entrySet()`

Ejemplo:

```
Iterator<Map.Entry<String, Integer>>it=mapa.entrySet().iterator();

while (it.hasNext()) {
    Map.Entry<String, Integer> entry = it.next();
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

RECORRIDO DE UN MAPA

El Iterator es especialmente útil para **eliminar elementos mientras se recorre**:

```
Iterator<Map.Entry<String, Integer>> it = mapa.entrySet().iterator();

while (it.hasNext()) {
    Map.Entry<String, Integer> entry = it.next();

    if (entry.getValue() < 10) {
        it.remove(); // forma correcta
    }
}
```

RECORRIDO DE UN MAPA

Ejemplo completo:

```
import java.util.*;
public class EjemploMap {
    public static void main(String[] args) {
        Map<String, Integer> edades = new HashMap<>();
        edades.put("Ana", 25);
        edades.put("Luis", 30);
        edades.put("Pedro", 20);
        for (Map.Entry<String, Integer> entry : edades.entrySet()) {
            System.out.println(entry.getKey() + " tiene " +
                               entry.getValue() + " años");
        }
        if (edades.containsKey("Ana")) {
            System.out.println("Ana está en el mapa");
        }
    }
}
```